Google Summer of Code
Project Proposal for Kiwix (OpenZIM)

Project Summary

Project: Refactor MWoffliner Core Architecture

Idea: My idea involves an all-round improvement to MWoffliner. **I want to improve maintainability, modularity, performance and most importantly testability by the end of GSoC**. The focus is on eliminating **circular dependencies (**I analyzed using **Depth First Search)**, restructuring the codebase into **clear layers with Dependency Injection (**breaking down few monolithic orchestrators**)**, implementing a **pipeline architecture**, and achieving **high test coverage**. The summary of changes in my proposed idea is on this page. All the details are explained later in the proposal.

Project Size:

- Small (90 Hours)
- Medium (175 Hours)
- **Large (350 Hours) ✓**

Contact Information:

- **Name:** Vikram Aditya Verma
- **Emai:** vikramadityaverma87@gmail.com
- **Github Username and URL:** VikramAditya33
- **Phone:** +91-6283621362
- **Address:** E-9 VATS VILLA, MANSAROVAR PARK, SHAHDARA, DELHI - 110032

About me

I am currently pursuing B.Tech in AI-DS from USAR GGSIPU in Delhi, India. As a first-year undergraduate, I have prior experience in full-stack development, working with web technologies including TypeScript, JavaScript (Node.js and NestJS), Python (FastAPI and Django), Vue.js, React.js, and Tailwind CSS. I am currently exploring system design and scalable architecture, recently working on a personal monorepo project utilizing uWebSockets, BullMQ queues and workers, Redis, and other technologies.

I have significant open source experience too, particularly with Kiwix and OpenZIM. My first contributions were to Kiwix Desktop and Kiwix JS, where Kelson and Jaifroid guided me through my initial open source work. Kelson later introduced me to Benoit, and I worked on the UI revamp of Project Gutenberg for the OpenZIM organization which has now been successfully merged into main branch. Through this experience, I gained hands-on knowledge of large-scale refactoring which would be significantly helpful in refactoring mwoffliner too.

Contributions to OpenZIM and Kiwix [Pull Requests]

1. **Pull Request #353 :** This PR implements Phase 1 and Phase 2 of the Gutenberg UI revamp: project structure reorganization and JSON file generation for the Vue.js frontend. This addresses issue #282 by replacing the SQLite database approach with a JSON-based data strategy that loads data on-demand, reducing memory usage on small devices.

2. **Pull Request #356:** This PR implements a complete Vue.js 3 frontend for the Project Gutenberg ZIM file, providing a modern, responsive user interface with full offline functionality. The implementation includes a Single Page Application (SPA) with comprehensive views, components, and a no-JavaScript fallback system.

3. **Pull Request #371:** This PR implements the CI/QA wiring for the new Vue.js UI and aligns the project's workflows with the patterns used in other openZIM projects (Mindtouch, freecodecamp). It also adds Prettier and ESLint to pre-commit and ensures Vitest is correctly wired with a minimal smoke test.

4. **Pull Request #377:** This PR filter authors to only include those with books.

5. **Pull Request #373:** This PR restores customization buttons for HTML book pages with a modern implementation using inline SVG icons, custom CSS, and JavaScript. Replaces the old Font Awesome-based buttons that were broken after the UI revamp.

6. **Pull Request #371:** This PR implements the CI/QA wiring for the new Vue.js UI and aligns the project's workflows with the patterns used in other openZIM projects (Mindtouch, freecodecamp). It also adds Prettier and ESLint to pre-commit and ensures Vitest is correctly wired with a minimal smoke test.

7. **Pull Request #378:** This PR removed the meta refresh tag entirely and replaced it with a simple link in the *<noscript>* block. This allows Kiwix-JS to properly intercept the link click using its existing *parseAnchorsJQuery()* mechanism.

8. **Pull Request #379:** This PR implements CLI flags for customizing primary and secondary colors in the Gutenberg ZIM builds, making them more flexible and themable. The colors are stored in *config.json* and applied to the Vue.js UI theme.

9. **Pull Request #390:** This PR updates all JS/CSS dependencies to their latest versions as of February 2026. This includes major version bumps for Vue Router (v5), Pinia (v3), Vite (v7), ESLint (v9), and Vitest (v4), along with a complete migration to ESLint's new flat config format.

10. **Pull Request #391:** This PR fixes the mobile view issues reported in issue #358 and improves code maintainability by introducing theme constants for consistent styling.

11. **Pull Request #393:** This PR cleans up the leftover files from UI migration.

12. **Pull Request #395:** This PR addresses issue #389 by making the language and sort filters collapsible, significantly reducing the visual space they occupy.

13. **Pull Request #398:** This PR Implemented internationalization support with *vue-i18n*, allowing users to switch the interface language across all supported languages. Converted all locale files from *jQuery* format to flat JSON structure with proper fallback handling.

14. **Pull Request #402:** This PR updates to Python 3.14, Node.js 24, and upgrade dependencies.

15. **Pull Request #403:** This PR brings display readable language names instead of language codes throughout the UI

16. **Pull Request #371:** This PR implements the CI/QA wiring for the new Vue.js UI and aligns the project's workflows with the patterns used in other openZIM projects (Mindtouch, freecodecamp). It also adds Prettier and ESLint to pre-commit and ensures Vitest is correctly wired with a minimal smoke test.

17. **Pull Request #408, #404, #405:** These PR fixes the Infobox assets that are exported first to fail fast if there's an issue, and the logs now accurately reflect the order of operations.

18. **Pull Request #409:** This PR **r**efactors the *Book.cover_page* field to improve code clarity and type correctness.

19. **Pull Request #411:** This PR improves the search results by enriching Vue.js index content and exclude *noJS* pages from search index.

20. **Pull Request #412:** This PR updated README.md, CONTRIBUTING.md and Documents in scraper/docs also deleted roadmap.md which was a reference document to keep the migration well documented.

21. **Pull Request #414:** This PR removed the *try-except* blocks that were preventing exceptions from reaching *@backoff.on_exception* decorator that already retries any *requests.exceptions.RequestException* (including from the RDF download). Changed *download_and_parse_book_rdf()* to let exceptions propagate instead of catching them and returning *None*.

22. **Pull Request #415:** This PR added Friulian language code mapping: *fur (IETF)* → *fvr (ISO 639-3)*

23. **Pull Request #416:** This PR adds a check after book processing to ensure at least one book was successfully downloaded before continuing with ZIM creation. If no books were downloaded, the scraper will fail with a clear error message instead of a cryptic *IndexError*.

24. **Pull Request #1370:** This PR fixed the gesture detection on short pages for Kiwix JS website where it replaced dynamic distance requirement (*innerHeight / 1.5*) with fixed 50px threshold.

25. **Pull Request #1377:** This PR fixes popover functionality of Kiwix JS to work with the *libzim* backend by adding conditional routing and proper data type handling.

26. **Pull Request #1389:** This PR fix ensures proper state tracking and *iframe* attachment during link-based navigation in Service Worker + *libzim* mode for Kiwix JS.

27. **Pull Request #1391:** This PR removed the legacy Wikivoyage workaround for image map popovers in Kiwix JS.

28. **Pull Request #1414:** This PR fixes issue #1253: Windows debug build assertion *'index.isValid()'* when search suggestion popup is triggered. The issue was caused by calling *QStyledItemDelegate::paint()* with an invalid *QModelIndex()*, which triggers Qt assertions in debug builds for Kiwix Desktop.

29. **Pull Request #430:** This PR implements automated CI checks for i18n translation files to catch common translation errors early for Gutenberg.

30. **Pull Request #442:** This PR rewrites the existing HTML to use *.webp* extension for converted images.

31. **Pull Request #441:** This PR fixes the i18n validation script failures and warnings.

32. **Pull Request #447:** This PR adds frontend unit tests for utility functions.

33. **Pull Request #451:** This PR deduplicates cover images by using the HTML version available.

Contributions to OpenZIM and Kiwix [Issues Opened & Assigned]

1. **Issue:** Kiwix search shows duplicate results from noJS fallback pages and Vue.js UI **#410**

2. **Issue:** Update to Python 3.14, Node.js 24, and upgrade all dependencies **#401**

3. **Issue:** Infobox assets logs is misleading **#400**

4. **Issue:** New UI: Languages and "Sort By" components takes too much visual space **#389**

5. **Issue:** New UI: Add back customization of the HTML version **#370**

6. **Issue:** Update README.md **#367**

7. **Issue:** Cleanup left-overs from new UI migration **#366**

8. **Issue:** No-JS fallback UI is not loading into Kiwix-JS restricted mode **#364**

9. **Issue:** Customizing primary and secondary colors **#363**

10. **Issue:** Implement Github workflows for Vue.JS UI **#362**

11. **Issue:** Wrong ZIM title **#360**

12. **Issue:** Authors without books should not make it to the ZIM **#359**

13. **Issue:** Refine new UI on small screens **#358**

14. **Issue:** New UI: Languages and "Sort By" components takes too much visual space **#389**

15. **Issue:** Fail the scraper run when RDF fails to download **#355**

16. **Issue:** Get rid of the search field inside the ZIM **#350**

17. **Issue:** Problem with the rating of books, seems always zero **#348**

18. **Issue:** Design and provide a reasonable access to books when Javascript is disabled **#345**

19. **Issue:** Do not continue ZIMing if all books have failed downloading **#338**

20. **Issue:** Add more language code mapping **#336**

21. **Issue:** Introduce dark theme **#334**

22. **Issue:** Display user-facing message when book is not found **#294**

23. **Issue:** Book card layout breaks with long title, author, or rating **#291**

24. **Issue:** Definition for current book Interface **#290**

25. **Issue:** Add buttons to view book directly from listing, without going through book details **#287**

26. **Issue:** Settle on a plan in terms of UI/UX for the new UI **#286**

27. **Issue:** Clicking anywhere on the tile must open the book details **#285**

28. **Issue:** HTML version of books must be displayed "inline" **#284**

29. **Issue:** Back/Forward navigation in browser history creates blank pages in some Zimit2 and other highly dynamic ZIMs **#1371**

30. **Issue:** In zimit2 ZIMs, pages that do not have an overflow cannot swipe away the menu and nav bar **#1266**

31. **Issue:** Variable *expectedArticleURLToBeDisplayed* is not updated when following links using libzim backend **#1385**

32. **Issue:** Remove legacy Wikivoyage workaround in popovers.js **#1387**

33. **Issue:** Sliding away toolbars erratic with Zimit classic archives when using libzim **#1380**

34. **Issue:** Make popover retrieval functions work with libzim **#1336**

35. **Issue:** In a Windows debug build search suggestion popup triggers a Qt assertion **#1253**

36. **Issue:** Enhanced Reading Experience for Gutenberg ZIM with Accessibility Features **#431**

37. **Issue:** Fix the translation errors caught by i18n validation script **#440**

38. **Issue:** Add frontend integration tests for views **#446**

39. **Issue:** Add frontend component tests for components **#445**

40. **Issue:** Add frontend unit tests for composables **#444**

41. **Issue:** Add frontend unit tests for utility functions **#443**

Overview of Changes Proposed According to My Idea:

1. **Introduce Dependency Injection and Remove Singletons:** I am proposing to eliminate all singleton patterns (Downloader, MediaWiki, RedisStore, RenderingContext) and replace them with constructor-based dependency injection. This will allow multiple instances to coexist, enable proper unit testing without infrastructure dependencies, and make dependencies explicit.

2. **Break Up Monolithic Files and Improve Code Quality:** I am proposing to refactor large files that violate the Single Responsibility Principle. The main orchestrator (*mwoffliner.lib.ts*, 749 lines) will be split into focused modules, *util/misc.ts* (568 lines) will be broken into specific utility modules, and *saveArticles.ts* (516 lines) will be decomposed. This also addresses code quality issues I identified, including reducing cyclomatic complexity, eliminating code duplication (repeated configuration patterns, duplicated test setup code), fixing dangerous patterns like singleton builders with mutable state that cause race conditions, and centralizing scattered configuration logic.

3. **Implement Storage Abstraction with Testing Infrastructure:** I am proposing to create storage interfaces (IArticleStore, IFileStore, IRedirectStore) with multiple implementations. The Redis implementation will be used in production, while an in-memory implementation will enable testing without infrastructure dependencies. This decouples the codebase from Redis, makes it possible to write fast unit tests, and addresses the 8 circular dependency cycles I detected using a DFS-based test by applying dependency inversion principles. This is critical for achieving the maintainer's goal of making the codebase more testable.

4. **Restructure Directory Organization:** I am proposing a minimal directory restructure that adds four new folders (core, services, storage, http) while keeping most existing files in place. This structure separates concerns clearly: core for orchestration and interfaces, services for business logic, storage for data persistence, and http for external communication. This creates clear boundaries between article fetching logic and ZIM file writing, which are currently mixed together.

5. **Optimize Processing Pipeline:** I am proposing to optimize the article processing pipeline by enabling concurrent file downloads as articles are processed, rather than waiting for all articles to complete first. This addresses the performance bottleneck where file downloading happens sequentially after article processing, and includes improvements to error handling consistency throughout the pipeline.

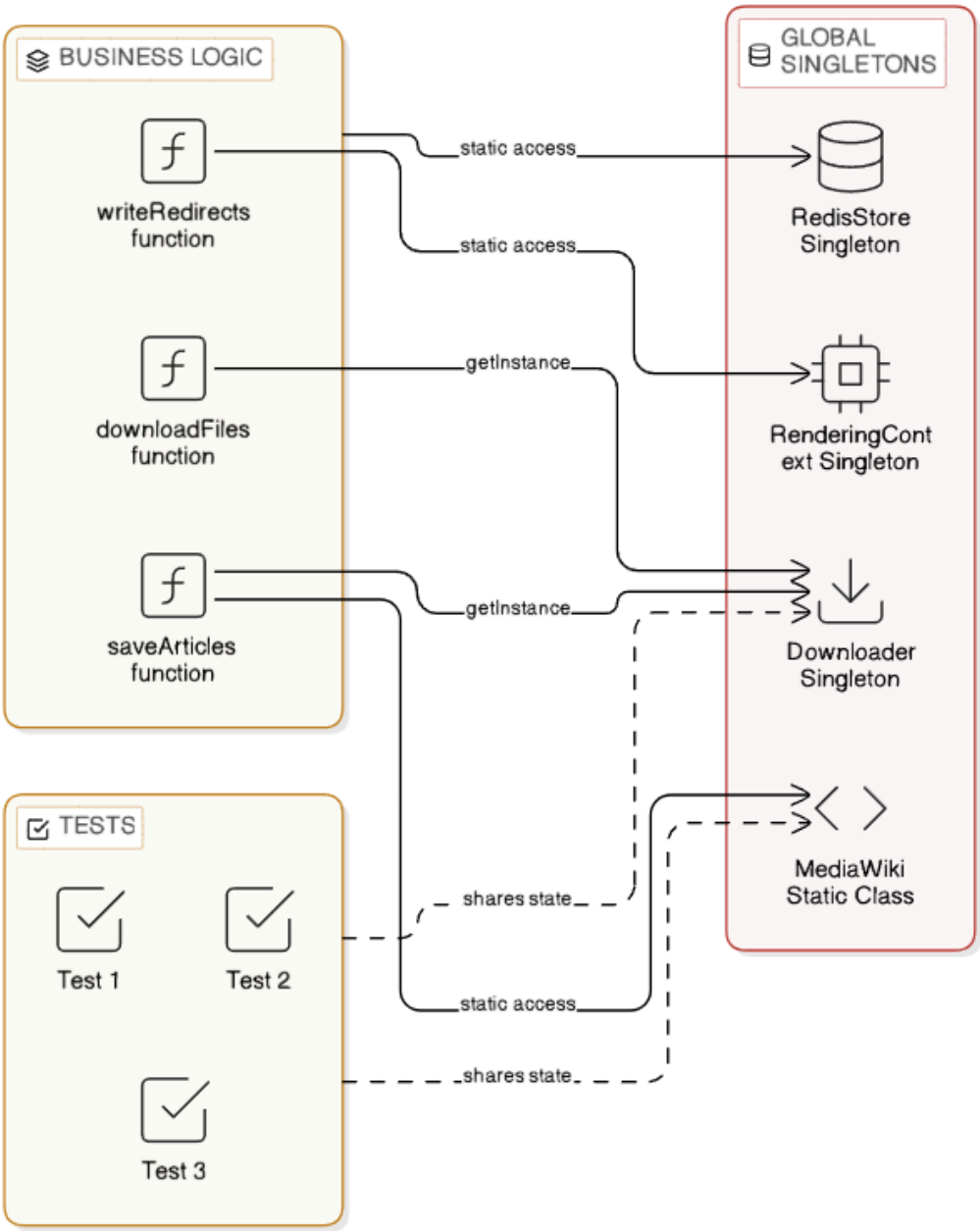
6. **Maintain Backward Compatibility:** I am proposing to keep the existing API functional by maintaining the old singleton/static methods as deprecated facades. The execute() function will continue to work but internally use the new architecture. This ensures existing users are not broken while providing a clear migration path.

Detailed Proposal Description

1) Introduce Dependency Injection and Remove Singletons

1.1 Introduction: Currently, mwoffliner uses singleton patterns and static classes throughout the codebase for core components like Downloader, MediaWiki, RedisStore, and RenderingContext. This creates global state that makes testing difficult, prevents running multiple scrapes in parallel, and creates tight coupling between modules. I propose to eliminate all singleton patterns and replace them with constructor-based dependency injection, making dependencies explicit and enabling proper unit testing without infrastructure dependencies.

1.2 Current Problems: Current Architecture: Singleton Pattern with Global State



Problems:

- All modules access the same singleton instances
- Tests share global state and interfere with each other
- Cannot run multiple scrapes in parallel
- Tight coupling to concrete implementations

Problem 1: Global State Prevents Test Isolation

The Downloader singleton maintains global state that must be manually reset between tests:

```
// Current: Every test must reset global state
beforeEach(() => {
 Downloader.init = {
 uaString: 'test-agent',
 speed: 1,
 reqTimeout: 60000
 }
})
```

This causes test pollution where one test's configuration affects another, making tests brittle and preventing parallel test execution.

Problem 2: Singleton Pattern Prevents Parallel Scraping

The singleton pattern prevents running multiple scrapes in parallel:

```
// Current: Both scrapes share the same Downloader instance
const scrape1 = execute({ mwUrl: 'en.wikipedia.org' })
const scrape2 = execute({ mwUrl: 'fr.wikipedia.org' })
// They interfere with each other
```

This wastes resources on multi-core machines and prevents scraping multiple small wikis in parallel (they must be scraped sequentially).

Problem 3: Tight Coupling Makes Testing Require Full Infrastructure

Because singletons are accessed globally throughout the codebase, every test requires Redis, network access, and full infrastructure setup:

```
// Current: Testing article rendering requires everything
describe('saveArticles', () => {
 beforeAll(startRedis) // Need Redis

 test('Article processing', async () => {
  const articles = await Downloader.getArticleDetailsIds(['London']) //
Network call
  await RedisStore.articleDetailXId.setMany(articles) // Redis
  await saveArticles(zimCreator, dump)
 })
})
```

1.3 Step-by-Step Approach:

Step 1: Create Core Interfaces

Create interface definitions in src/core/interfaces/ that define contracts for all major components:

```
// src/core/interfaces/IDownloader.ts
export interface IDownloader {
 download(url: string): Promise<Buffer>
 downloadImage(url: string, width?: number):
Promise<Buffer>
 getArticle(articleId: string): Promise<string>
}
```

```
// src/core/interfaces/IMediaWikiClient.ts
export interface IMediaWikiClient {
 getBaseUrl(): string
 getArticleDetails(articleIds: string[]):
Promise<ArticleDetail[]>
 hasActionParseApi(): Promise<boolean>
}
```

```
// src/core/interfaces/IArticleStore.ts
export interface IArticleStore {
 get(id: string): Promise<ArticleDetail>
 set(id: string, article: ArticleDetail): Promise<void>
 exists(id: string): Promise<boolean>
}
```

Step 2: Convert Downloader from Singleton to Injectable Class

Modify *src/Downloader.ts* to remove singleton pattern and accept dependencies via constructor:

```
/// Before: Singleton pattern
class Downloader {
 private static instance: Downloader
 public static getInstance() {
  if (!this.instance) {
   this.instance = new Downloader()
  }
  return this.instance
 }
}
```

```
// After: Constructor injection
export class HttpDownloader implements IDownloader {
 constructor(
  private config: DownloaderConfig,
  private logger: ILogger
 ) {}

 async download(url: string): Promise<Buffer> {
  this.logger.info(`Downloading ${url}`)
  // implementation
 }
}
```

Step 3: Convert MediaWiki from Static Class to Instance Class

Transform MediaWiki from a static class with global state to an instance class:

```
// Before: Static class with global state
class MediaWiki {
 static base: string
 static actionApiPath: string

 static async getArticle(id: string) {
  const url = this.base + this.actionApiPath
  // uses global state
 }
}
```

```
// After: Instance class with injected dependencies
export class MediaWikiClient implements IMediaWikiClient {
 constructor(
  private config: MediaWikiConfig,
  private downloader: IDownloader
 ) {}

 getBaseUrl(): string {
  return this.config.baseUrl
 }

 async getArticle(id: string): Promise<string> {
  const url = this.config.baseUrl + this.config.apiPath
  return this.downloader.download(url)
 }
}
```

Step 4: Convert RedisStore from Singleton to Injectable Service

Refactor RedisStore to accept configuration via constructor and implement storage interfaces:

```
// Before: Singleton with global access
class RedisStore {
 private static instance: RedisStore
 public static articleDetailXId: RedisKvs<ArticleDetail>

 private constructor() {
  this.articleDetailXId = new RedisKvs('article')
 }
}
```

```
/// After: Injectable service implementing interfaces
export class RedisArticleStore implements IArticleStore {
 private kvs: RedisKvs<ArticleDetail>

 constructor(config: RedisConfig) {
  this.kvs = new RedisKvs('article', config)
 }

 async get(id: string): Promise<ArticleDetail> {
  return this.kvs.get(id)
 }
}
```

Step 5: Convert RenderingContext from Singleton to Injectable Service

Refactor RenderingContext to remove singleton pattern and accept dependencies via constructor:

```
// Before: Singleton pattern
class RenderingContext {
 private static instance: RenderingContext
 public static renderers: Map<string, Renderer>

 static async createRenderers(rendererName: string) {
  if (!this.instance) {
   this.instance = new RenderingContext()
  }
  // Initialize renderers with global state
 }
}
```

```
// After: Injectable service
export class RenderingService {
 private renderers: Map<string, IRenderer>

 constructor(
  private downloader: IDownloader,
  private mediaWiki: IMediaWikiClient,
  private config: RenderingConfig
 ) {
  this.renderers = new Map()
 }

 async createRenderers(rendererName: string):
Promise<void> {
  // Initialize renderers with injected dependencies
  const renderer = new ActionParseRenderer(
   this.downloader,
   this.mediaWiki,
   this.config
  )
  this.renderers.set(rendererName, renderer)
 }

 getRenderer(name: string): IRenderer {
  return this.renderers.get(name)
 }
}
```

Step 6: Update All Callers to Use Dependency Injection

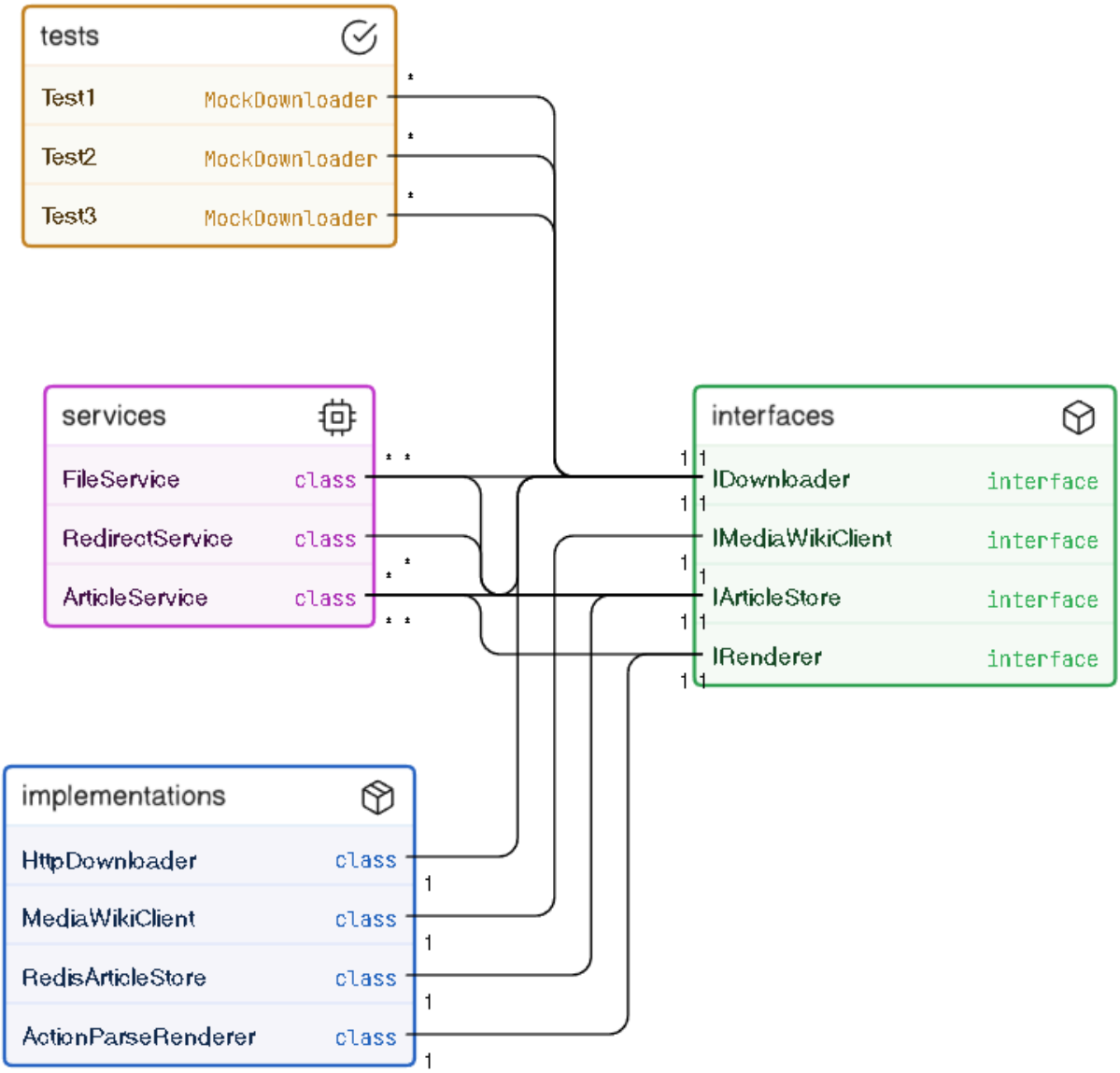
Update the main orchestrator and all modules to use constructor injection instead of accessing singletons:

```
// Before: Accessing global singletons
async function saveArticles(zimCreator, dump) {
 const downloader = Downloader.getInstance()
 const articles = await RedisStore.articleDetailXId.getAll()
 const renderer = RenderingContext.renderers.get('default')
 // ...
}
```

```
// After: Dependencies injected
class ArticleService {
 constructor(
  private downloader: IDownloader,
  private articleStore: IArticleStore,
  private renderingService: RenderingService,
  private logger: ILogger
 ) {}

 async saveArticles(zimCreator: ZimCreator, dump: Dump) {
  const articles = await this.articleStore.getAll()
  const renderer =
this.renderingService.getRenderer('default')
  // ...
 }
}
```

1.4 Backward Compatibility Strategy

New Architecture: Dependency Injection with Interfaces:



Benefits:

- Each service receives its own instances via constructor
- Tests create isolated instances (no shared state)
- Multiple scrapes can run in parallel with separate instances
- Loose coupling through interfaces (dependency inversion)

To maintain backward compatibility during the transition, will keep the old singleton APIs as deprecated facades:

```
// Downloader.ts - Maintain old API
export class HttpDownloader implements IDownloader {
 // New injectable implementation
}
```

```
// Deprecated singleton facade for backward compatibility
/** @deprecated Use HttpDownloader with dependency
injection instead */
export class Downloader {
 private static instance: HttpDownloader

 static getInstance(): HttpDownloader {
  if (!this.instance) {
   this.instance = new HttpDownloader(defaultConfig,
defaultLogger)
  }
  return this.instance
 }
}
```

This allows existing code to continue working while new code uses dependency injection.

2) Break Up Monolithic Files and Improve Code Quality

2.1 Introduction: The codebase contains several monolithic files that violate the Single Responsibility Principle. The main orchestrator in src/mwoffliner.lib.ts is 648 lines with complexity over 50, src/util/saveArticles.ts is 516 lines handling multiple concerns, and src/util/misc.ts is 568 lines serving as a utility junk drawer. I propose to break these files into smaller, focused modules, reducing cyclomatic complexity to under 10 per function, eliminating code duplication (repeated configuration patterns, duplicated test setup code), fixing dangerous patterns like singleton builders with mutable state that cause race conditions, and centralizing scattered configuration logic into a single ScraperConfig class.

2.2 Current Problems:

Problem 1: 749-Line God File in Main Orchestrator

The execute() function in src/mwoffliner.lib.ts handles everything from argument parsing to ZIM creation:

```
// Current: One massive function doing everything
async function execute(argv: any) {
 // 749 lines handling:
 // Argument parsing and validation
 // MediaWiki initialization
 // Redis setup and connection
 // Article list fetching
 // Category processing
 // Article downloading and rendering
 // File downloading
 // Redirect processing
 // ZIM file creation
 // Error handling and logging
 // Cleanup and shutdown
}
```

This function has quite high cyclomatic complexity, making it extremely difficult to understand, test, or modify.

Problem 2: Mixed Responsibilities in saveArticles.ts

The src/util/saveArticles.ts file (516 lines) mixes article processing, file extraction, HTML manipulation, and storage operations:

```
// Current: One file doing too many things
export async function saveArticles(zimCreator, dump) {
 // Fetch articles from storage
 // Parse HTML and extract dependencies
 // Rewrite URLs
 // Minify HTML
 // Extract image dependencies
 // Check file resolutions
 // Update Redis with file info
 // Save to ZIM file
 // Handle errors and logging
}
```

Problem 3: Utility Drawer with 568 Lines

The src/util/misc.ts file has become a catch-all for utilities:

```
// Current: Everything thrown into one file
export function downloadAndSaveModule() { }
export function getMediaBase() { }
export function getSizeFromUrl() { }
export function getFullUrl() { }
// more functions
```

This makes it difficult to understand what utilities are available and creates unnecessary dependencies.

Problem 4: Code Duplication at several Configurations and also in Test Setup

The same configuration patterns are repeated throughout the codebase which violates DRY principle and makes configuration changes require updates at several locations.

2.3 Step-by-Step Approach:

Step 1: Extract Configuration Management from Main Orchestrator

Create a dedicated ScraperConfig class to handle all configuration logic:

```
// src/core/ScraperConfig.ts
export class ScraperConfig {
 constructor(private argv: any) {}

 validate(): void {
  // Validate all arguments
  if (!this.argv.mwUrl) {
   throw new Error('mwUrl is required')
  }
 }

 getDownloaderConfig(): DownloaderConfig {
  return {
   uaString: this.argv.userAgent,
   speed: this.argv.speed,
   reqTimeout: this.argv.requestTimeout * 1000
  }
 }

 getMediaWikiConfig(): MediaWikiConfig {
  return {
   baseUrl: this.argv.mwUrl,
   apiPath: this.argv.apiPath
  }
 }
}
```

Step 2: Create Service Layer for Business Logic

Extract business logic into focused service classes:

```
// src/services/ArticleService.ts
export class ArticleService {
 constructor(
  private articleStore: IArticleStore,
  private renderer: IRenderer,
  private logger: ILogger
 ) {}

 async processArticle(articleId: string):
Promise<ProcessedArticle> {
  const article = await this.articleStore.get(articleId)
  const rendered = await this.renderer.render(article.html)
  return { id: articleId, html: rendered }
 }
}
```

```
// src/services/FileService.ts
export class FileService {
 constructor(
  private fileStore: IFileStore,
  private downloader: IDownloader
 ) {}

 async extractFilesFromHtml(html: string):
Promise<FileDetail[]> {
  // Extract image and media file dependencies
 }

 async downloadFile(file: FileDetail): Promise<Buffer> {
  return this.downloader.download(file.url)
 }
}
```

Step 3: Refactor Main Orchestrator into Smaller Functions

Break the 648-line execute() function into focused, testable functions:

```
// src/core/Scraper.ts
export class Scraper {
 constructor(
  private config: ScraperConfig,
  private articleService: ArticleService,
  private fileService: FileService,
  private redirectService: RedirectService
 ) {}

 async run(): Promise<void> {
  await this.initialize()
  const articles = await this.fetchArticleList()
  await this.processArticles(articles)
  await this.downloadFiles()
  await this.processRedirects()
  await this.finalize()
 }

 private async initialize(): Promise<void> {
  // Setup logic
 }

 private async fetchArticleList(): Promise<string[]> {
  // Article list fetching
 }

 private async processArticles(articles: string[]):
Promise<void> {
  // Article processing
 }
}
```

Each method has a single responsibility and low cyclomatic complexity.

Step 4: Split saveArticles.ts into Focused Modules

Break the monolithic saveArticles.ts into separate concerns:

```
// src/services/ArticleProcessor.ts
export class ArticleProcessor {
 async process(article: ArticleDetail):
Promise<ProcessedArticle> {
  const html = await this.fetchHtml(article)
  const optimized = await this.optimizeHtml(html)
  return { id: article.id, html: optimized }
 }
}
```

```
// src/services/HtmlOptimizer.ts
export class HtmlOptimizer {
 rewriteUrls(html: string): string { }
 minifyHtml(html: string): string { }
 extractDependencies(html: string): Dependency[] { }
}
```

```
// src/services/ZimWriter.ts
export class ZimWriter {
 async writeArticle(article: ProcessedArticle):
Promise<void> {
  // Write to ZIM file
 }
}
```

Step 5: Organize util/misc.ts by Concern

Split the 568-line utility file into focused modules:

```
// src/util/url.ts - URL utilities only
export function getFullUrl(base: string, path: string): string { }
export function parseMediaUrl(url: string): ParsedUrl { }
export function getSizeFromUrl(url: string): number { }
```

```
// src/util/validation.ts - Validation utilities only
export function isValidEmail(email: string): boolean { }
export function isValidUrl(url: string): boolean { }
```

```
// src/util/html.ts - HTML utilities only
export function getStrippedTitleFromHtml(html: string): string { }
export function genHeaderCSSLink(css: string): string { }
export function genHeaderScript(script: string): string { }
```

```
// src/util/filesystem.ts - File system utilities only
export function ensureDirectoryExists(path: string): void { }
export function getFileExtension(filename: string): string { }
```

Step 6: Eliminate Code Duplication with Centralized Configuration

Create configuration helpers to eliminate repeated patterns:

```
// src/core/ScraperConfig.ts (enhanced)
export class ScraperConfig {
 // Centralized configuration logic
 getDownloaderConfig(): DownloaderConfig {
  return {
   uaString: `${this.argv.userAgent} (${this.argv.adminEmail})`,
   speed: this.argv.speed || 1,
   reqTimeout: (this.argv.requestTimeout || 60) * 1000,
   webp: this.argv.webp !== false,
   optimisationCacheUrl: this.argv.optimisationCacheUrl || ''
  }
 }
}
```

```
// test/helpers/TestConfig.ts - Shared test configuration
export class TestConfig {
 static createDefaultDownloaderConfig(): DownloaderConfig {
  return {
   uaString: 'test-agent (test@example.com)',
   speed: 1,
   reqTimeout: 60000,
   webp: true,
   optimisationCacheUrl: ''
  }
 }

 static createDefaultMediaWikiConfig(): MediaWikiConfig {
  return {
   baseUrl: 'https://en.wikipedia.org',
   apiPath: '/w/api.php'
  }
 }
}
```

```
// Usage in tests - no more duplication:
beforeEach(() => {
 const config = TestConfig.createDefaultDownloaderConfig()
 downloader = new HttpDownloader(config, logger)
})
```

Step 7: Fix Singleton Builder Pattern with Immutable Builders

Replace unsafe mutable singleton builders with immutable instance builders:

```
// Before: Singleton with mutable state (not a good practice)
export class UrlBuilder {
 private static instance: UrlBuilder
 private currentBase: string

 static getInstance(): UrlBuilder { }
 setBase(base: string): this { this.currentBase = base; return this }
}
```

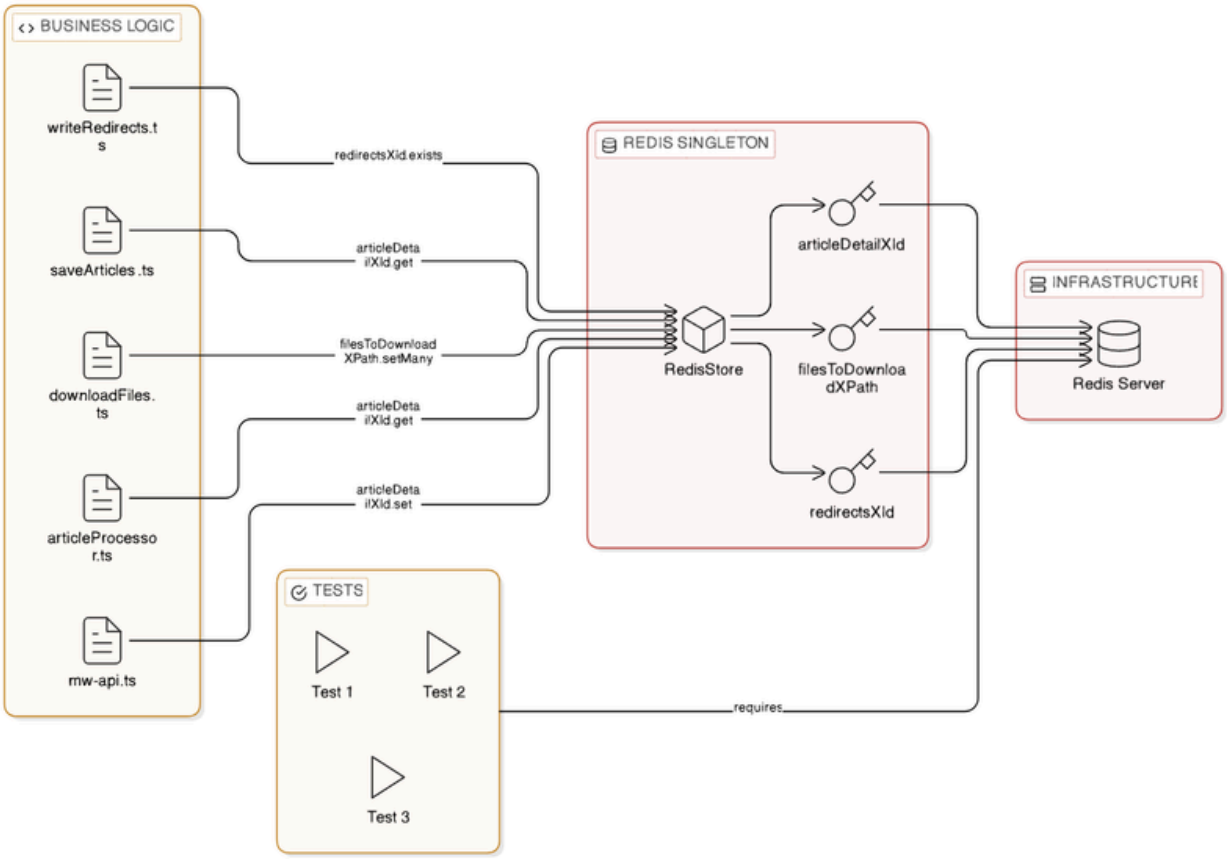
```
// After: Immutable instance builder (safe and good practice)
export class UrlBuilder {
 constructor(private readonly base: string) {}

 withPath(path: string): UrlBuilder {
  // Returns new instance, doesn't mutate
  return new UrlBuilder(this.base + path)
 }

 build(): string {
  return this.base
 }
}
```

```
// Usage:
const builder1 = new UrlBuilder('https://en.wikipedia.org')
const builder2 = new UrlBuilder('https://fr.wikipedia.org')
const url1 = builder1.withPath('/api.php').build()
const url2 = builder2.withPath('/api.php').build()
// Each builder is independent
```

3) Implement Storage Abstraction and Improve Testing

3.1 Introduction: Currently, the codebase has direct dependencies on Redis throughout, with too many direct Redis calls scattered across modules. This makes it very difficult to run tests without a Redis server and prevents using alternative storage implementations. Additionally, the current test structure with tests separated from source code makes it difficult to maintain and find related tests. I propose to create storage abstraction interfaces (IArticleStore, IFileStore, IRedirectStore) with multiple implementations including Redis and in-memory for testing, and reorganize tests to be co-located with source code using `.spec.ts` files for unit tests and `.integration.ts` files for integration tests. This decouples the codebase from testing only using Redis, enables true unit tests that run in milliseconds without infrastructure dependencies, and addresses the 8 circular dependency cycles I detected using a DFS-based test by applying dependency inversion principles. This is critical for making the codebase more testable.

3.2 Current Problems: Current Architecture: Direct Redis Dependencies



Problems:

- Too many direct Redis calls scattered across codebase
- All tests require Redis server running
- Very difficult to test without infrastructure
- Tight coupling to Redis implementation
- Creates circular dependencies

Problem 1: Tests Require Redis Infrastructure

Test in the test/unit/ directory requires a running Redis server:

```
// Current: All tests need Redis
describe('saveArticles', () => {
 beforeAll(startRedis) // Start Redis server
 afterAll(stopRedis)  // Stop Redis server

 test('Article processing', async () => {
  await RedisStore.articleDetailXId.set('article1', data)
  await saveArticles(zimCreator, dump)
  const result = await RedisStore.articleDetailXId.get('article1')
 })
})
```

This makes tests slow (7+ minutes for full suite), brittle (fail if Redis unavailable), and difficult to run in CI environments.

Problem 2: Hardcoded Redis Throughout Codebase

Redis is accessed directly at many locations:

```
// Current: Direct Redis access everywhere
await RedisStore.articleDetailXId.get(articleId)
await RedisStore.filesToDownloadXPath.setMany(files)
await RedisStore.redirectsXId.exists(redirectId)
```

This tight coupling prevents swapping storage implementations and makes testing require full infrastructure.

Problem 3: Edge Cases Require Complex Test Setup

Testing error scenarios or edge cases requires complex Redis state manipulation:

```
// Current: Complex setup to test error handling
test('handles missing article', async () => {
 await startRedis()
 await RedisStore.articleDetailXId.flush()
 // Manually set up Redis state to simulate missing article
 await RedisStore.articleDetailXId.set('article1', data)
 // Now can test
 await expect(processArticle('article2')).rejects.toThrow()
})
```

With storage abstraction, this becomes trivial.

Problem 4: Circular Dependencies Prevent Modular Testing

The codebase has 8 circular dependency cycles detected by a DFS-based test. Analysis of the dependency graph reveals the scale and nature of the coupling problem:

Dependency graph metrics:
- 43 source files with 155 total dependencies
- Average 3.6 dependencies per file
- But top files have 10-16 imports each

Most problematic files (by import count):
- mwoffliner.lib.ts: 16 imports (god file orchestrating everything)
- util/saveArticles.ts: 15 imports (monolithic business logic)
- renderers/abstract.renderer.ts: 13 imports (base class with too many dependencies)
- Downloader.ts: 11 imports (HTTP client coupled to business logic)
- MediaWiki.ts: 10 imports (API client coupled to business logic)

Most tightly coupled modules (imported by many files):
- Downloader.ts: imported by 12 files (singleton creates tight coupling)
- MediaWiki.ts: imported by 11 files (singleton creates tight coupling)
- util/misc.ts: imported by 10 files (junk drawer creates unnecessary dependencies)

8 circular dependency cycles detected:

1. *Downloader → abstract.renderer → MediaWiki → mw-api → Downloader*
2. *Downloader → abstract.renderer → MediaWiki → Downloader*
3. *Downloader → api.director → url.builder → misc → Downloader*
4. *MediaWiki → mw-api → MediaWiki*
5. *Downloader → abstract.renderer → Downloader*
6. *Downloader → abstract.renderer → dump → Downloader*
7. *Downloader → abstract.renderer → articles → Downloader*
8. *Downloader → error.manager → Downloader*

These cycles exist because modules directly import concrete implementations instead of depending on interfaces. For example:

```
// src/util/saveArticles.ts
import Downloader from '../Downloader' // Direct dependency
import { RedisStore } from '../RedisStore' // Direct dependency
// This creates tight coupling and circular dependencies
```

The singleton patterns for Downloader and MediaWiki are particularly problematic - they're imported by 12 and 11 files respectively, creating a web of dependencies that makes modular testing very hard. Storage abstraction with dependency inversion breaks these cycles by having modules depend on interfaces instead of concrete implementations.

Problem 5: Test Structure Makes Maintenance Difficult

The current test structure separates tests from source code, making it hard to find and maintain tests:

```
Current structure:
src/
 ├── Downloader.ts        # Source file
 ├── util/
 │    └── saveArticles.ts      # Source file

test/
 ├── unit/
 │    ├── downloader.test.ts    # Test far from source
 │    └── saveArticles.test.ts   # Test far from source
```

Problems:

- Tests are far from the code they test (different directory tree)
- Hard to find related tests when working on a file
- Easy to forget to update tests when changing code
- When refactoring, tests don't move with their source files
- No clear distinction between true unit tests and integration tests (all in `test/unit/`)
- Can't easily run only fast tests during development

```
Better approach:
src/
 ├── Downloader.ts
 ├── Downloader.spec.ts      # Unit test right next to source
 ├── Downloader.integration.ts   # Integration test clearly marked
 ├── util/
 │    ├── saveArticles.ts
 │    └── saveArticles.spec.ts   # Test co-located with source
```

This makes tests visible, easy to find, and encourages writing them.

3.3 Step-by-Step Approach:

Step 1: Define Storage Interfaces

Create clear interface contracts for all storage operations:

```
// src/core/interfaces/IArticleStore.ts
export interface IArticleStore {
 get(id: string): Promise<ArticleDetail | null>
 set(id: string, article: ArticleDetail): Promise<void>
 getMany(ids: string[]): Promise<Map<string, ArticleDetail>>
 setMany(articles: Map<string, ArticleDetail>): Promise<void>
 exists(id: string): Promise<boolean>
 existsMany(ids: string[]): Promise<Map<string, boolean>>
 flush(): Promise<void>
}
```

```
// src/core/interfaces/IFileStore.ts
export interface IFileStore {
 get(path: string): Promise<FileDetail | null>
 set(path: string, file: FileDetail): Promise<void>
 getMany(paths: string[]): Promise<Map<string, FileDetail>>
 setMany(files: Map<string, FileDetail>): Promise<void>
}
```

```
// src/core/interfaces/IRedirectStore.ts
export interface IRedirectStore {
 get(id: string): Promise<RedirectDetail | null>
 set(id: string, redirect: RedirectDetail): Promise<void>
 iterateItems(batchSize: number, callback: (items: Map<string,
RedirectDetail>) => Promise<void>): Promise<void>
}
```

Step 2: Implement Redis Storage Adapter

Refactor existing RedisStore to implement the interfaces:

```
// src/storage/RedisArticleStore.ts
export class RedisArticleStore implements IArticleStore {
 private kvs: RedisKvs<ArticleDetail>

 constructor(config: RedisConfig) {
  this.kvs = new RedisKvs('article', config)
 }

 async get(id: string): Promise<ArticleDetail | null> {
  return this.kvs.get(id)
 }

 async setMany(articles: Map<string, ArticleDetail>): Promise<void> {
  return this.kvs.setMany(articles)
 }

 // Implement all interface methods
}
```

Step 3: Create In-Memory Storage Implementation for Testing

Build a simple in-memory implementation that matches the interface:

```
// src/storage/InMemoryArticleStore.ts
export class InMemoryArticleStore implements IArticleStore {
 private data = new Map<string, ArticleDetail>()
 async get(id: string): Promise<ArticleDetail | null> {
  return this.data.get(id) || null
 }
 async set(id: string, article: ArticleDetail): Promise<void> {
  this.data.set(id, article)
 }
 async getMany(ids: string[]): Promise<Map<string, ArticleDetail>> {
  const result = new Map<string, ArticleDetail>()
  for (const id of ids) {
   const article = this.data.get(id)
   if (article) result.set(id, article)
  }
  return result
 }
 async flush(): Promise<void> {
  this.data.clear()
 }
}
```

This implementation is fast (no network calls), simple (just a Map), and perfect for unit tests.

Step 4: Update All Code to Use Storage Interfaces

Replace direct Redis access with injected storage interfaces:

```
// Before: Direct Redis access
async function saveArticles(zimCreator, dump) {
 const articles = await RedisStore.articleDetailXId.getAll()
 // ...
}
```

```
// After: Injected storage interface
class ArticleService {
 constructor(private articleStore: IArticleStore) {}

 async saveArticles(zimCreator: ZimCreator, dump: Dump) {
  const articles = await this.articleStore.getAll()
  // ...
 }
}
```

Step 5: Rewrite Tests to Use In-Memory Storage

Transform slow integration tests into fast unit tests:

```
// Before: Slow test requiring Redis (40+ seconds)
describe('saveArticles', () => {
 beforeAll(startRedis)
 afterAll(stopRedis)

 test('processes articles', async () => {
  await RedisStore.articleDetailXId.set('article1', data)
  await saveArticles(zimCreator, dump)
 })
})
```

```
// After: Fast unit test with in-memory storage (milliseconds)
describe('ArticleService', () => {
 test('processes articles', async () => {
  const store = new InMemoryArticleStore()
  await store.set('article1', data)
  const service = new ArticleService(store)
  await service.saveArticles(zimCreator, dump)
  expect(/* assertions */)
 })
})
```

S**tep 6**: Break Circular Dependencies with Dependency Inversion

Apply dependency inversion to eliminate the 8 circular dependency cycles:

```
// Before: Direct imports create circular dependencies
// saveArticles.ts → Downloader → abstract.renderer → saveArticles
(CYCLE!)
import Downloader from '../Downloader'
import { RedisStore } from '../RedisStore'

export async function saveArticles(zimCreator, dump) {
 const downloader = Downloader.getInstance()
 const articles = await RedisStore.articleDetailXId.getAll()
}
```

```
// After: Depend on interfaces, no circular dependencies
import { IDownloader } from '../core/interfaces/IDownloader'
import { IArticleStore } from '../core/interfaces/IArticleStore'

export class ArticleService {
 constructor(
  private downloader: IDownloader,
  private articleStore: IArticleStore
 ) {}

 async saveArticles(zimCreator: ZimCreator, dump: Dump) {
  const articles = await this.articleStore.getAll()
  // No circular dependency - depends on interface, not
implementation
 }
}
```

Step 7: Reorganize Test Structure for Better Modularity

Migrate from centralized test directory to co-located test files for better maintainability:

```
// Current structure: Tests far from source
test/
 ├── unit/
 │   ├── downloader.test.ts    // Far from src/Downloader.ts
 │   ├── saveArticles.test.ts   // Far from src/util/saveArticles.ts
 │   └── ...
```

```
// New structure: Tests co-located with source
src/
├── services/
│   ├── ArticleService.ts
│   ├── ArticleService.spec.ts     // Unit test (fast, with mocks)
│   ├── ArticleService.integration.ts // Integration test (with Redis)
│   ├── FileService.ts
│   └── FileService.spec.ts
├── storage/
│   ├── InMemoryArticleStore.ts
│   ├── InMemoryArticleStore.spec.ts  // Fast unit test
│   ├── RedisArticleStore.ts
│   └── RedisArticleStore.integration.ts // Requires Redis
├── http/
│   ├── HttpDownloader.ts
│   ├── HttpDownloader.spec.ts     // Unit test with mock axios
│   └── HttpDownloader.integration.ts // Real network calls
└── util/
    ├── url.ts
    └── url.spec.ts        // Pure function tests
test/
└── e2e/
    ├── en.e2e.test.ts
    └── ...
```

Benefits of co-located tests:

- Tests are immediately visible next to the code they test
- Easy to find and update tests when changing code
- Refactoring moves tests with their source files
- Clear test type separation (`.spec.ts` vs `.integration.ts`)
- Industry standard (used by NestJS, Angular, React, Vue)
- Encourages writing tests (they're visible and accessible)

After completing this refactoring:

- Unit tests run in milliseconds without infrastructure
- Test suite execution time will reduce
- Tests can run in any environment without Redis setup
- Easy to test edge cases and error scenarios
- Storage implementation can be swapped without code changes
- Clear separation between business logic and infrastructure
- All 8 circular dependency cycles eliminated through dependency inversion
- Codebase is more testable, achieving maintainer's primary goal
- Test coverage will be increased
- Tests are co-located with source code for better maintainability
- Clear distinction between unit tests (`.spec.ts`) and integration tests (`.integration.ts`)
- Easy to run only fast tests during development

4) Restructure Directory Organization

4.1 Introduction: The current flat directory structure mixes concerns and makes it difficult to understand the codebase architecture. Files like Downloader.ts, MediaWiki.ts, and RedisStore.ts sit alongside utility files and rendering code with no clear organization. I propose a minimal directory restructure that adds four new folders (core, services, storage, http) while keeping most existing files in place to minimize disruption. This structure separates concerns clearly: core for orchestration and interfaces, services for business logic, storage for data persistence, and http for external communication. This creates clear boundaries between article fetching logic and ZIM file writing, which are currently mixed together.

4.2 Current Problems

Problem 1: Flat Structure Hides Architecture

All source files are in src/ with no organization:

```
src/
├── Downloader.ts
├── MediaWiki.ts
├── RedisStore.ts
├── mwoffliner.lib.ts   (Main orchestrator)
├── DOMUtils.ts
├── Logger.ts
├── Templates.ts
├── Dump.ts
├── renderers/
└── util/       (Everything else)
```

The architecture is not immediately clear from the directory structure, making it difficult to find related files.

Problem 2: util/ Folder is a Junk Drawer

The util/ directory contains too many files with no clear organization:

```
src/util/
├── articles.ts
├── saveArticles.ts
├── downloadFiles.ts
├── misc.ts       (500+ lines of everything)
├── RedisKvs.ts     (Storage infrastructure)
├── RedisQueue.ts    (Storage infrastructure)
├── mw-api.ts
├── url.helper.ts
└── ...
```

Problem 3: No Clear Dependency Direction

Without directory structure, it's unclear which modules depend on which.

Problem 4: Article Fetching and ZIM Writing Are Mixed Together

The current structure doesn't separate concerns between fetching articles from MediaWiki and writing to ZIM files:

```
// src/util/saveArticles.ts - Everything mixed together
export async function saveArticles(zimCreator, dump) {
 // Fetch from MediaWiki
 const articles = await Downloader.getArticleDetailsIds(articleIds)

 // Process HTML
 const html = await renderer.render(article.html)

 // Write to ZIM
 await zimCreator.addArticle(article)

 // Extract files for downloading
 const files = extractFiles(html)

 // All concerns mixed in one file
}
```

This makes it hard to understand where data comes from (MediaWiki) versus where it goes (ZIM file), and prevents reusing the fetching logic separately from the writing logic.

4.3 Step-by-Step Approach

Step 1: Create Core Directory for Orchestration

Create src/core/ for main orchestration and configuration:

```
src/core/
 ├── Scraper.ts       (Main orchestrator, refactored from
mwoffliner.lib.ts)
 ├── ScraperConfig.ts    (Configuration management)
 └── interfaces/      (All interface definitions)
   ├── IDownloader.ts
   ├── IArticleStore.ts
   ├── IFileStore.ts
   ├── IRedirectStore.ts
   ├── IMediaWikiClient.ts
   ├── IRenderer.ts
   └── IZimWriter.ts
```

Step 2: Create Services Directory for Business Logic

Create src/services/ for business logic extracted from util/:

```
src/services/
 ├── ArticleService.ts   (Article processing logic)
 ├── FileService.ts      (File handling logic)
 ├── RedirectService.ts  (Redirect handling logic)
 ├── ArticleProcessor.ts (HTML processing)
 └── HtmlOptimizer.ts    (HTML optimization)
```

This separates concerns clearly:

```
// src/services/ArticleService.ts - Fetching logic
export class ArticleService {
 constructor(
  private mediaWiki: IMediaWikiClient,
  private articleStore: IArticleStore
 ) {}

 async fetchArticles(articleIds: string[]): Promise<void> {
  // Only responsible for fetching from MediaWiki
  const articles = await this.mediaWiki.getArticleDetails(articleIds)
  await this.articleStore.setMany(articles)
 }
}
```

```
// src/services/ZimWriterService.ts - Writing logic
export class ZimWriterService {
 constructor(
  private articleStore: IArticleStore,
  private zimCreator: ZimCreator
 ) {}

 async writeArticlesToZim(): Promise<void> {
  // Only responsible for writing to ZIM
  const articles = await this.articleStore.getAll()
  for (const article of articles) {
   await this.zimCreator.addArticle(article)
  }
 }
}
// Clear separation: fetching vs writing
```

Step 3: Create Storage Directory for Storage Implementations

Create src/storage/ for all storage-related code:

```
src/storage/
 ├── RedisArticleStore.ts  (Redis implementation of IArticleStore)
 ├── RedisFileStore.ts   (Redis implementation of IFileStore)
 ├── RedisRedirectStore.ts (Redis implementation of IRedirectStore)
 ├── InMemoryArticleStore.ts (In-memory for testing)
 ├── InMemoryFileStore.ts  (In-memory for testing)
 └── RedisKvs.ts      (Existing Redis utilities)
```

Step 4: Create HTTP Directory for HTTP Clients

Create src/http/ for HTTP and API clients:

```
src/http/
 ├── HttpDownloader.ts   (Refactored Downloader.ts)
 ├── MediaWikiClient.ts   (Refactored MediaWiki.ts)
 └── MediaWikiApi.ts    (Extracted from util/mw-api.ts)
```

Step 5: Clean Up Utilities Directory

Reorganize src/util/ to contain only pure utility functions:

```
src/util/
 ├── url.ts        (URL utilities from misc.ts)
 ├── validation.ts     (Validation utilities from misc.ts)
 ├── html.ts       (HTML utilities from misc.ts)
 ├── filesystem.ts     (File system utilities from misc.ts)
 ├── Timer.ts       (Existing)
 └── const.ts       (Existing)
```

5) Optimize Article Processing Pipeline

5.1 Introduction: The current architecture processes articles sequentially in distinct phases: first all articles are fetched and processed, then all files are downloaded, then redirects are written. This sequential approach creates a performance bottleneck where file downloading happens only after all article processing completes, wasting network bandwidth and time. I propose implementing a streaming pipeline architecture where articles flow through processing stages (fetch, render, extract files, save) with overlapping execution, enabling concurrent file downloads as articles are processed rather than waiting for all articles to complete first. This also includes improvements to error handling consistency throughout the pipeline.

5.2 Current Problems

Problem 1: Sequential Processing Wastes Time

The main orchestrator processes everything in sequence:

```
// Current: Everything happens one after another
await getArticleIds()        // Step 1: Get article list (MUST be first)
await getCategoriesForArticles()   // Step 2: Get categories (depends on Step 1)
await trimUnmirroredPages()      // Step 3: Trim pages (depends on Step 2)
await saveArticles(zimCreator, dump)  // Step 4: Process ALL articles (depends on Step 3)
await downloadFiles()        // Step 5: Download ALL files (BLOCKS on Step 4 completing!)
await writeArticleRedirects()     // Step 6: Write redirects (could run earlier)
```

What MUST be sequential:

- Steps 1-3 have data dependencies and must run in order
- Step 5 needs file metadata from Step 4

What CAN be parallelized:

- Step 4 (article processing) and Step 5 (file downloading) can overlap using a streaming pipeline
- Step 6 (redirects) is independent and could run concurrently with file downloads

The main bottleneck is that file downloading waits for all article processing to complete, when files could be downloaded as soon as they're discovered during article processing.

Problem 2: Performance Bottleneck - Files Downloaded After All Articles Processed

The sequential architecture creates a performance bottleneck where files are only downloaded after every article has been processed:

```
// Current flow - PERFORMANCE BOTTLENECK:
// 1. Process article 1 → extract files → store in Redis
// 2. Process article 2 → extract files → store in Redis
// ...
// 3. Process article N → extract files → store in Redis
// 4. NOW start downloading files from Redis (BOTTLENECK)
await saveArticles(zimCreator, dump)  // Processes all articles
await downloadFiles(filesToDownloadXPath, zimCreator, dump) // Then downloads all files
// Problem: Network bandwidth sits idle during article processing!
```

Network bandwidth sits completely idle during article processing, and all file metadata accumulates in Redis waiting to be downloaded. Files are discovered during article processing but not downloaded until all articles finish. This is the primary performance bottleneck in the scraping process.

Why this is a bottleneck:

- Article processing is CPU-bound (parsing HTML, rewriting URLs)
- File downloading is network-bound (waiting for HTTP responses)
- These operations use different resources and could run concurrently
- With a streaming pipeline, files could download while articles are still being processed

Problem 3: Manual Promise Queue Management

The saveArticles function uses complex manual promise queue management:

```
// Current: Manual promise queue (error-prone)
const parsePromiseQueue: [string, Promise<Error>][] = []

for (const [articleId, articleDetail] of Object.entries(articles)) {
 rets = await Downloader.getArticle(...)
 promises.push([articleId, saveArticle(...)])

 if (parsePromiseQueue.length) {
  const [articleId, parsePromise] = parsePromiseQueue.shift()
  const err = await parsePromise
  // Handle error...
 }

 if (promises.length) {
  parsePromiseQueue.push(flattenPromises(promises))
 }
}
```

This is a bit complex, hard to maintain, and doesn't maximize parallelism.

5.3 Step-by-Step Approach

Step 1: Design Pipeline Stage Interface

Create a simple interface for pipeline stages:

```
// src/core/interfaces/IPipelineStage.ts
export interface IPipelineStage<TInput, TOutput> {
 process(input: TInput): Promise<TOutput>
 getName(): string
}
```

```
// src/core/pipeline/ArticleContext.ts
export interface ArticleContext {
 articleId: string
 articleDetail?: ArticleDetail
 html?: string
 renderedHtml?: string
 files?: FileDetail[]
 saved?: boolean
}
```

Step 2: Implement Individual Pipeline Stages

Create focused stages for each processing step:

```
// src/core/pipeline/stages/FetchStage.ts
export class FetchStage implements IPipelineStage<ArticleContext,
ArticleContext> {
 constructor(private mediaWiki: IMediaWikiClient) {}

 async process(context: ArticleContext): Promise<ArticleContext> {
  const html = await this.mediaWiki.getArticle(context.articleId)
  return { ...context, html }
 }

 getName(): string { return 'Fetch' }
}
```

```
// src/core/pipeline/stages/RenderStage.ts
export class RenderStage implements IPipelineStage<ArticleContext,
ArticleContext> {
 constructor(private renderer: IRenderer) {}

 async process(context: ArticleContext): Promise<ArticleContext> {
  const renderedHtml = await this.renderer.render(context.html)
  return { ...context, renderedHtml }
 }

 getName(): string { return 'Render' }
}
```

```
// src/core/pipeline/stages/ExtractFilesStage.ts
export class ExtractFilesStage implements
IPipelineStage<ArticleContext, ArticleContext> {
 constructor(private fileService: FileService) {}

 async process(context: ArticleContext): Promise<ArticleContext> {
  const files = await
this.fileService.extractFilesFromHtml(context.renderedHtml)
  // Queue files for download immediately
  files.forEach(f => this.fileService.queueForDownload(f))
  return { ...context, files }
 }

 getName(): string { return 'ExtractFiles' }
}
```

```
// src/core/pipeline/stages/SaveStage.ts
export class SaveStage implements IPipelineStage<ArticleContext,
ArticleContext> {
 constructor(private zimWriter: IZimWriter) {}

 async process(context: ArticleContext): Promise<ArticleContext> {
  await this.zimWriter.writeArticle(context.articleId,
context.renderedHtml)
  return { ...context, saved: true }
 }

 getName(): string { return 'Save' }
}
```

Step 3: Create Pipeline Orchestrator

Build a pipeline that chains stages together:

```
// src/core/pipeline/ArticlePipeline.ts
export class ArticlePipeline {
 private stages: IPipelineStage<ArticleContext, ArticleContext>[] = []
 addStage(stage: IPipelineStage<ArticleContext, ArticleContext>): this {
  this.stages.push(stage)
  return this
 }
 async execute(articleId: string): Promise<ArticleContext> {
  let context: ArticleContext = { articleId }
  for (const stage of this.stages) {
   context = await stage.process(context)
  }
  return context
```

Step 4: Implement Streaming Processing with Controlled Concurrency

Process articles in a stream with proper concurrency control:

```
// src/core/pipeline/ArticlePipeline.ts (continued)
import pLimit from 'p-limit'
export class ArticlePipeline {
 async executeStream(
  articleIds: string[],
  concurrency: number = 8
 ): Promise<void> {
  const limit = pLimit(concurrency)
  const results = await Promise.all(
   articleIds.map(articleId =>
    limit(async () => {
     try {
      await this.execute(articleId)
     } catch (error) {
      this.logger.error(`Failed to process ${articleId}:`, error)
     }
    })
   )
  )
 }
}
// Usage in Scraper
const pipeline = new ArticlePipeline()
 .addStage(new FetchStage(mediaWiki))
 .addStage(new RenderStage(renderer))
 .addStage(new ExtractFilesStage(fileService))
 .addStage(new SaveStage(zimWriter))
await pipeline.executeStream(articleIds, 8)
```

Step 5: Implement Parallel File Download Service

Create a service that downloads files as soon as they're discovered.

Step 6: Implement Consistent Error Handling Strategy

Create a unified error handling approach throughout the pipeline.

6) Maintain Backward Compatibility

6.1 Introduction: While refactoring the codebase, it is critical to maintain backward compatibility for existing users who depend on mwoffliner as a library or command-line tool. I propose implementing a compatibility layer that preserves the existing public API while internally using the new architecture, using deprecation warnings to guide users toward the new patterns, and providing clear migration documentation with code examples.

6.2 Step-by-Step Approach

Step 1: Maintain Old execute() Function as Facade

Keep the existing execute() function working while using new architecture internally.

Step 2: Keep Singleton APIs with Deprecation Warnings

Maintain old singleton patterns as facades to new implementations.

```
// src/Downloader.ts
import { HttpDownloader } from './http/HttpDownloader'
/**
 * @deprecated Use HttpDownloader with dependency injection instead.
 * This singleton pattern is maintained for backward compatibility only.
 *
 * Migration example:
 * // Old way:
 * const downloader = Downloader.getInstance()
 *
 * // New way:
 * const downloader = new HttpDownloader(config, logger)
 *
 */
```

Step 3: Provide Migration Examples in Documentation

Create comprehensive migration guide.

7) Optional Enhancements

If time permits after completing the core refactoring, the following optimization could provide additional performance benefits:

Redis Lua Script for Redirect Validation

The limitation enhancement: Redirect validation requires 2 Redis calls per redirect (check if redirect exists as article, check if target exists). This creates N×2 network round-trips where N is the number of redirects.

Current code:

```
// src/mwoffliner.lib.ts - writeArticleRedirects()
await redirectsXId.iterateItems(Downloader.speed, async (redirects) =>
{
 for (const [redirectId, { targetId, fragment }] of
Object.entries(redirects)) {
  // 2 Redis calls per redirect - network round-trip for each
  if (await RedisStore.articleDetailXId.exists(redirectId)) {
   logger.warn(`Skipping redirect - already exists as article`)
   continue
  }
  if (!(await RedisStore.articleDetailXId.exists(targetId))) {
   logger.warn(`Skipping redirect - target doesn't exist`)
   continue
  }
  zimCreator.addRedirection(redirectId, redirectTitle, targetId)
 }
})
```

Optimization with Lua Script:

Benefits:

- Reduces N×2 Redis calls to N/BATCH_SIZE calls (e.g., 2000 calls → 2 calls for 1000 redirects)
- All validation logic runs on Redis server (no network latency per redirect)
- Significantly faster redirect processing

```
// Lua script executes on Redis server, eliminating network round-trips
const validateRedirectsScript = `
 local articleHash = ARGV[1]
 local redirectIds = cjson.decode(ARGV[2])
 local targetIds = cjson.decode(ARGV[3])
 local validRedirects = {}

 for i, redirectId in ipairs(redirectIds) do
  local targetId = targetIds[i]

  -- Skip if redirect exists as article
  if redis.call('HEXISTS', articleHash, redirectId) == 1 then
   -- skip
  -- Skip if target doesn't exist
  elseif redis.call('HEXISTS', articleHash, targetId) == 0 then
   -- skip
  else
   table.insert(validRedirects, {redirectId, targetId})
  end
 end

 return cjson.encode(validRedirects)
`
```

```
// Batch validate redirects in chunks
const BATCH_SIZE = 1000
for (let i = 0; i < redirectIds.length; i += BATCH_SIZE) {
 const batch = redirectIds.slice(i, i + BATCH_SIZE)
 const validRedirects = await redis.eval(validateRedirectsScript, {
  arguments: [
   'articleDetail',
   JSON.stringify(batch),
   JSON.stringify(batch.map(id => redirectMap[id].targetId))
  ]
 })

 // Process valid redirects
 for (const [redirectId, targetId] of validRedirects) {
  zimCreator.addRedirection(redirectId, redirectTitle, targetId)
 }
}
```

Any Other Relevant Information

Project Outcomes

The successful completion of this refactoring project will deliver:

- **Enhanced Maintainability**
- **Improved Testability**
- **Better Developer Experience**
- **Foundation for Future Growth**
- **Better Resource Utilization**

Post-GSoC Commitment

I plan to continue contributing to OpenZIM organization as my contribution in Gutenberg2zim one is still going on. I'll also try to help onboard new contributors and introducing them to OpenZIM and Kiwix ecosystem as currently not too many people are contributing to this Open source repository, I feel like this is very underrated repository to contribute to like genuinely the people here are super cool and the repository itself is more cooler. In the end my goal is not just to complete a summer project, but to become a long-term contributor to the Kiwix ecosystem.

Availability and Time Commitment

I am well-positioned to dedicate the required time to this project:

May - June: I have end-semester examinations during this period. However, my university has a flexible attendance policy, allowing me to manage my time effectively. I can easily commit enough hours per week during this period.

Mid Jun - July End: Following my exams, I have a 1.5-month summer break where I can fully dedicate myself to the project.

I am confident that this time allocation, combined with effective communication with my mentors, will ensure the successful completion of all project deliverables within the GSoC timeline (Even before it maybe ^^).

Thank you so much once again OpenZIM and Kiwix for introducing me to Open Source, it really has taught me so many things that no other thing other than Open Source could teach me.